NVIC (Nested Vector Interrupt Controller) Register Description

Embedded Controller

Prof. Young-Keun Kim

NVIC Registers

- **NVIC** is used to manage interrupts
  - Enable, Disable, Set priority level
  - Arm Cortex M4: 52 maskable interrupt channel, 16 priority levels

[Programming Manual: Ch 4.3]

- **NVIC (Nested Vectored Interrupt Controller)**
  - Interrupt set-enable register: NVIC_ISERx
  - Interrupt clear-enable register: NVIC_ICERx
  - Interrupt set-pending register: NVIC_ISPRx
  - Interrupt clear-pending register: NVIC_ICPRx
  - Interrupt active bit register: NVIC_IABRx
  - Interrupt priority register: NVIC_IPRx
  - Software trigger interrupt register: NVIC_STIR

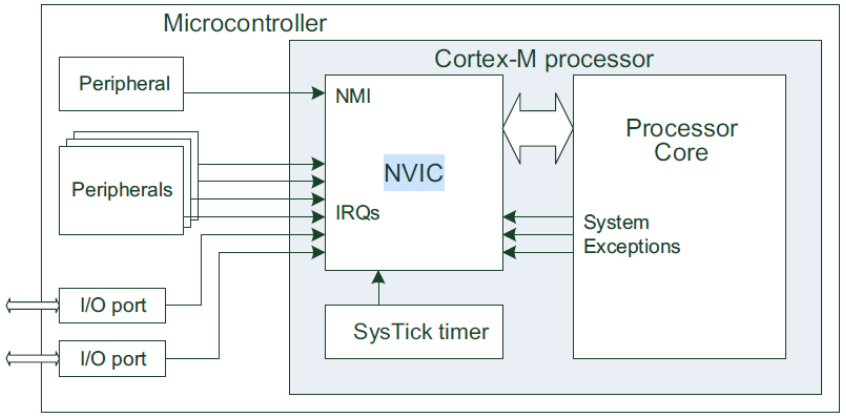


FIGURE 7.1
Various sources of exceptions in a typical microcontroller

Use CMSIS functions:

```
NVIC_DisableIRQ (IRQn);             // Disable interrupt
NVIC_EnableIRQ (IRQn);              // Enable interrupt
NVIC_ClearingPending (IRQn);        // clear pending status
NVIC_SetPriority (IRQn, priority); // set priority level
```


한동대학교 HANDONG GLOBAL UNIVERSITY

NVIC Registers

- **Interrupt set-enable register (NVIC_ISERx):**

[PM: Ch 4.3]

NVIC_ISER0 bits 0 to 31 are for interrupt 0 to 31, respectively

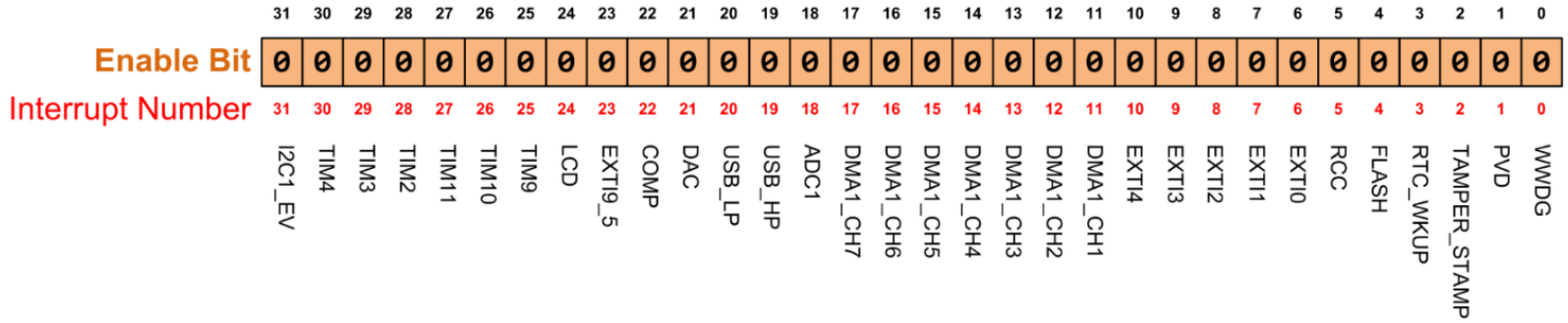
NVIC_ISER1 bits 0 to 31 are for interrupt 32 to 63, respectively

....

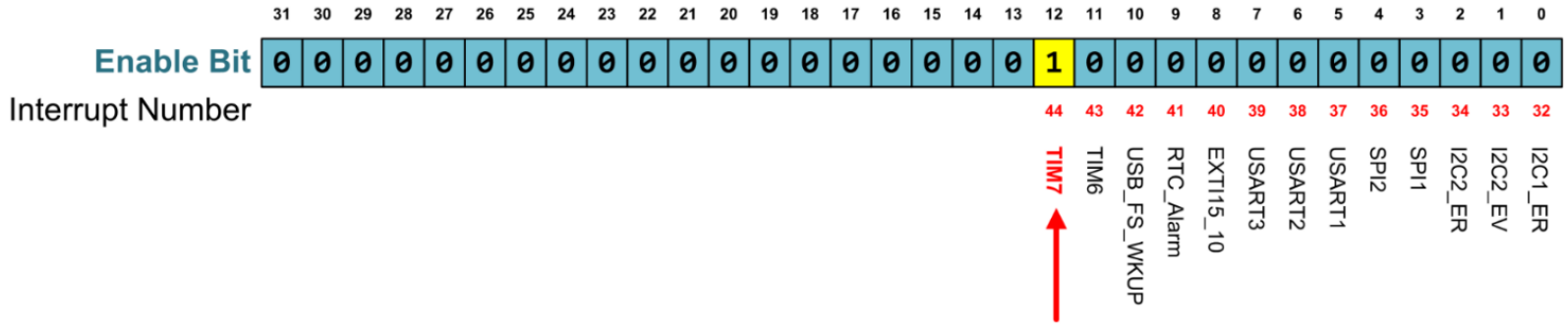
Write:
0: No effect
1: Enable interrupt

Read:
0: Interrupt disabled
1: Interrupt enabled.

Interrupt Set Enable Register 0 (ISER0)



Interrupt Set Enable Register 1 (ISER1) *Address of ISER1 = Address of ISER0 + 4*




```
TIM7_IRQn = 44
NVIC->ISER[1] = 1 << 12;     // Enable Timer 7 interrupt
```

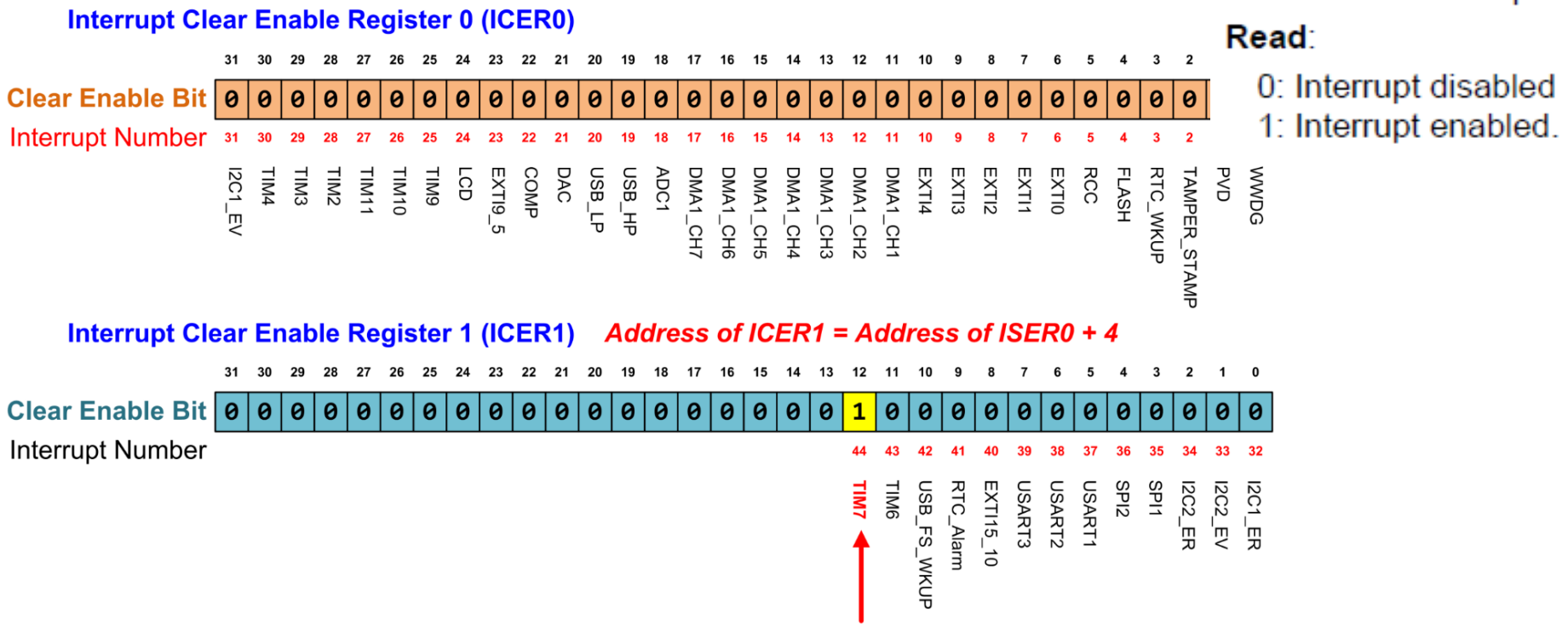

한동대학교 HANDONG GLOBAL UNIVERSITY

NVIC Registers

- **Interrupt clear-enable register (NVIC_ICERx)**
  - Writing '1' disables the chosen interrupt

Write:
0: No effect
1: Disable interrupt

Read:
0: Interrupt disabled
1: Interrupt enabled.




```
TIM7_IRQn = 44

NVIC->ICER[1] = 1 << 12;      // Disable Timer 7 interrupt
```


한동대학교 HANDONG GLOBAL UNIVERSITY

NVIC Registers

- Different methods of **Enable/Disable** peripheral interrupts: IRQn ≥ 0

Method 1: (use CMSIS functions)

- NVIC_EnableIRQ (IRQn); // Enable interrupt
- NVIC_DisableIRQ (IRQn); // Disable interrupt

Method 2: change registers with given IRQn (0~44)

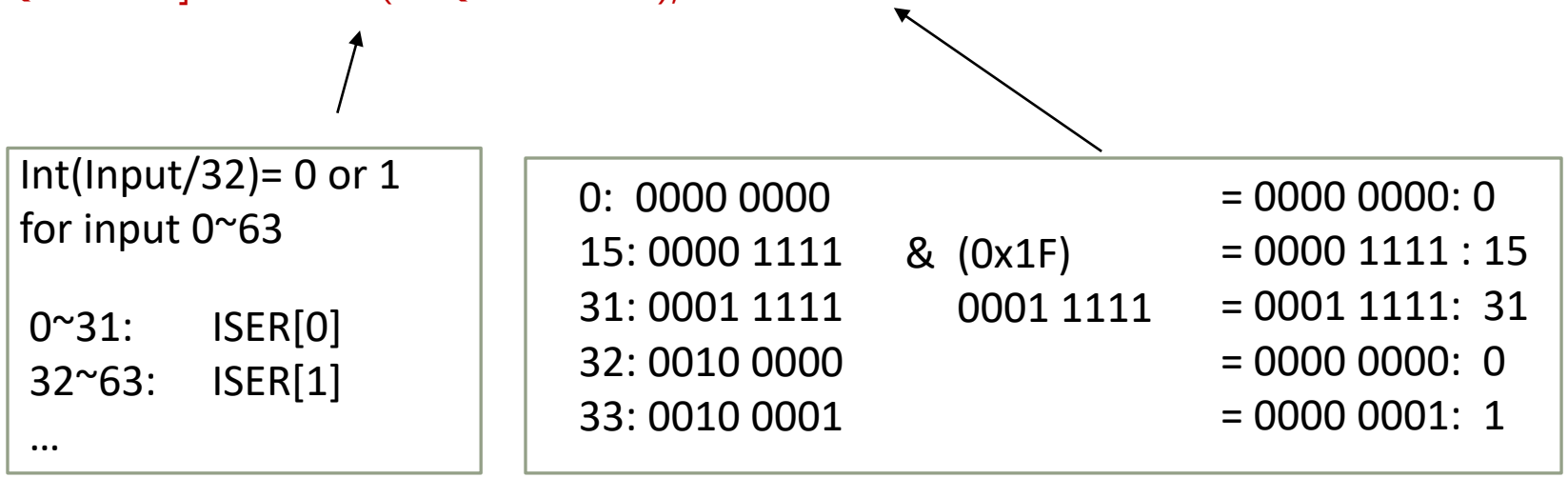
ISER[0], ISER[1]
ICER[0], ICER[1]

– Enable:

- NVIC->ISER[IRQn / 32] = 1 << (IRQn % 32); // or
- NVIC->ISER[IRQn >> 5] = 1 << (IRQn & 0x1F); // Better solution: 2^5=32

– Disable:

- NVIC->ICER[IRQn >> 5] = 1 << (IRQn & 0x1F);



한동대학교 HANDONG GLOBAL UNIVERSITY

NVIC Registers

- **Interrupt set/clear pending (NVIC_ISPRx, NVIC_ICPRx), x=0 to 7**
  - If MCU cannot process this interrupt immediately when an interrupt occurs
    - Pending bit is **Set** (waiting list)
  - If **Clear_pend** = H, then, it removes from pending list
  - What happens if interrupt is **disabled** but **pending is already set**?
    - Executes interrupt then disable it

Bits 31:0 **SETPEND**: Interrupt set-pending bits

Write:

0: No effect

1: Changes interrupt state to pending

Read:

0: Interrupt is not pending

1: Interrupt is pending

Bits 31:0 **CLRPEND**: Interrupt clear-pending bits

Write:

0: No effect

1: Removes the pending state of an interrupt

Read:

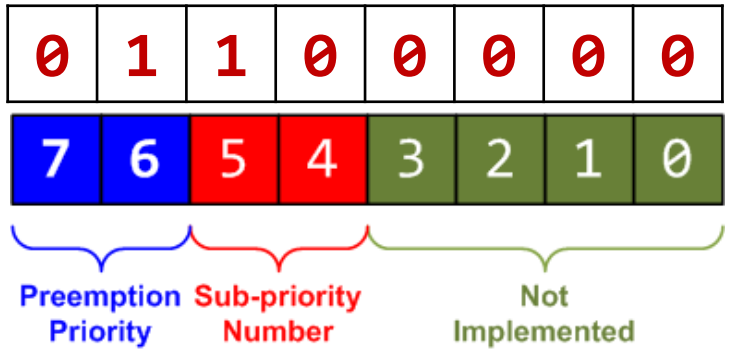
0: Interrupt is not pending

1: Interrupt is pending

한동대학교 HANDONG GLOBAL UNIVERSITY

NVIC Registers

- **Interrupt priority register (NVIC_IPRx, x=0 to 59)**
  - Priority: lower number has higher priority.
    - check the group priority (Preemption)
      -> then check the Sub-Priority
  - Uses 8-bits for each IRQ priority
    - IP [N] (N = 0 to 239) for each of the 240 interrupts.
  - But from 8-bits, it only uses 4 bits: [7:4].
    - bits[3:0] read as zero and ignore writes.



IP = 0x60 = 96

```
NVIC_SetPriority (IRQn, priority);
NVIC_SetPriority(7, 6); // IP=96


// or user defined
NVIC->IP[7] = (6 << 4) & 0xff; //IP[7]=96
```

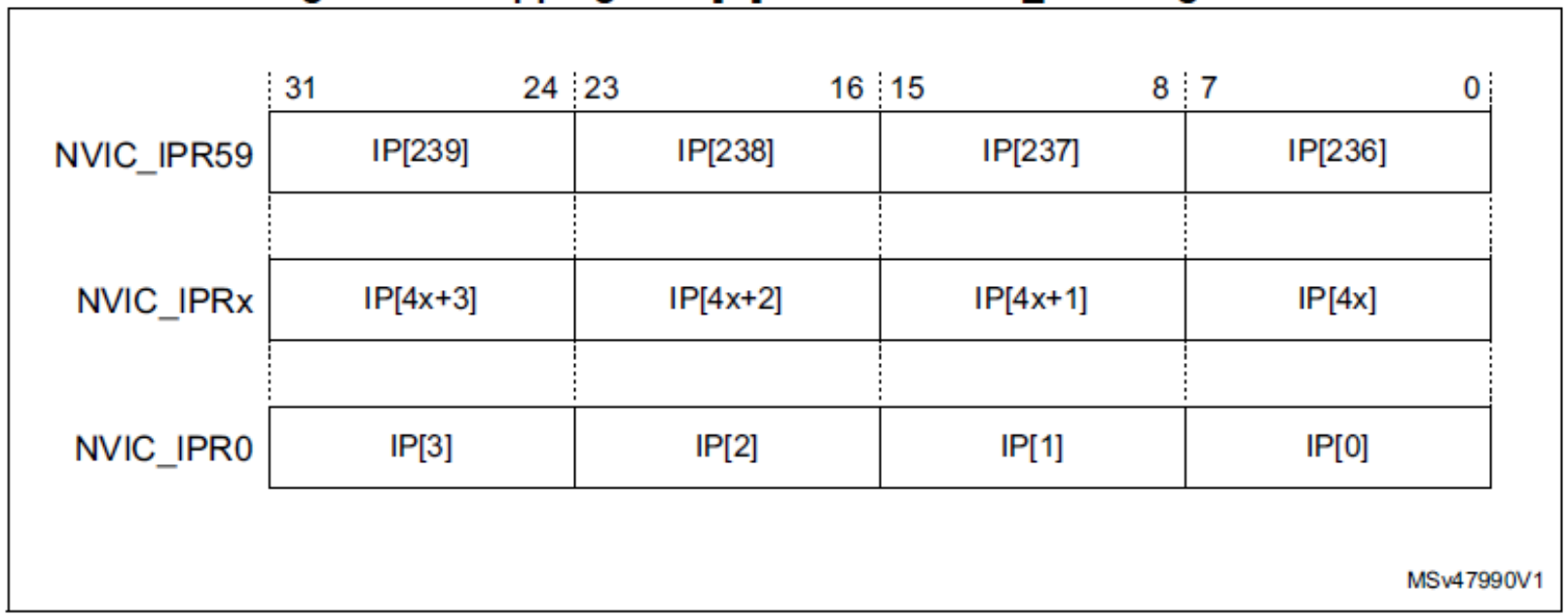

한동대학교 HANDONG GLOBAL UNIVERSITY

NVIC Registers

- **Interrupt priority register: (NVIC_IPRx): x=0 to 59**

The NVIC_IPRx (x = 0 to 59) byte-accessible registers provide 8-bit priority fields IP[N] (N = 0 to 239) for each of the 240 interrupts. Every register holds four IP[N] fields of the CMSIS interrupt priority array, as shown in *Figure 19*.

Figure 19. Mapping of IP[N] fields in NVIC_IPRx registers



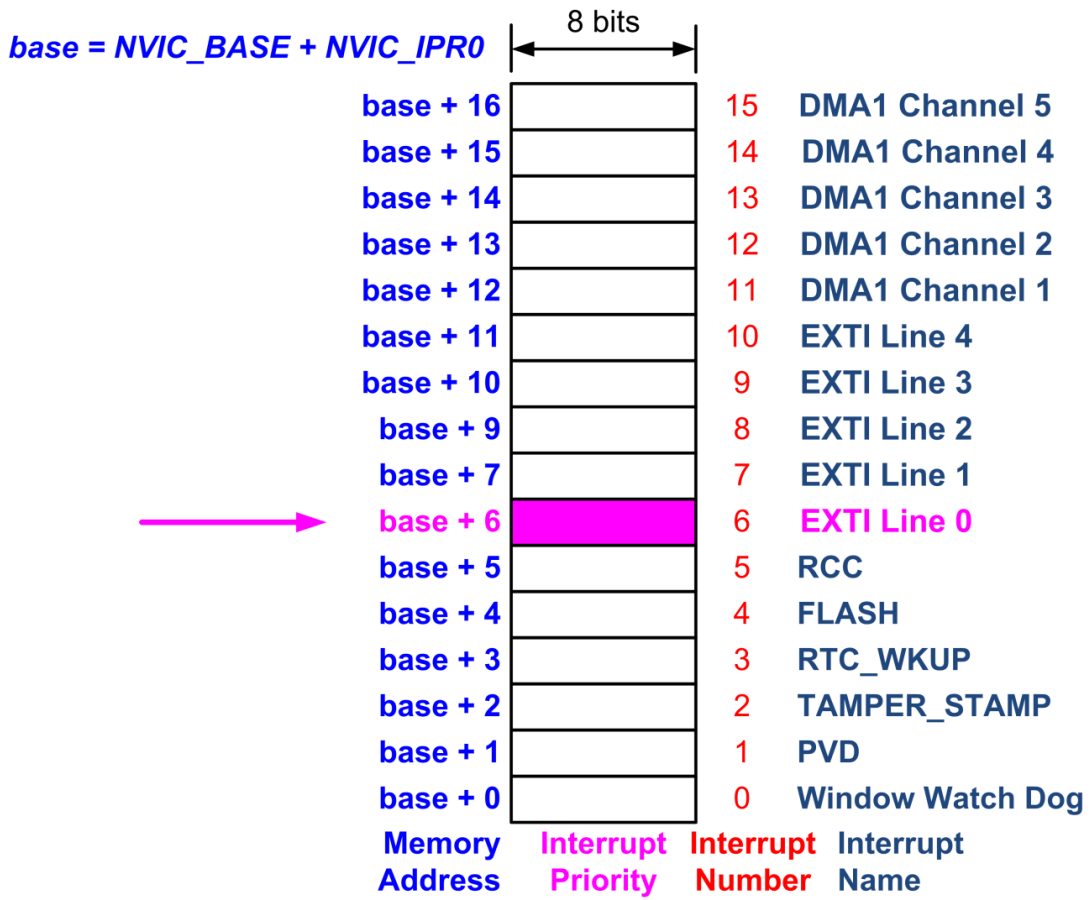
The following table shows the bit assignment of any NVIC_IPRx register. Each IP[N] field order can be expressed as N = 4 * x + byte offset.

한동대학교 HANDONG GLOBAL UNIVERSITY

NVIC Registers

- Interrupt priority register: (NVIC_IPRx): x=0 to 59

```
// Set the priority for EXTI 0 (Interrupt number 6) as priority 15
NVIC->IP[6] = 0xF0;
```



한동대학교 HANDONG GLOBAL UNIVERSITY

NVIC Registers

- **Programming example of Interrupt Priority Setting**

```
/**  core_cm4.h
typedef struct
{
        __IOM uint32_t ISER[8U];            /*  Interrupt Set Enable Register */
       uint32_t RESERVED0[24U];
        __IOM uint32_t ICER[8U];
         …
        __IOM uint8_t  IP[240U];           /* Interrupt Priority Register (8Bit wide) */
}  NVIC_Type;
…
#define NVIC_BASE     (SCS_BASE +  0x0100UL)                /*!< NVIC Base Address */
#define NVIC              ((NVIC_Type      *)     NVIC_BASE     )   /*!< NVIC configuration struct */


/**  main.c
NVIC->IP[7] = (6 << 4) & 0xff;                    //IP[7]=96

// or use CMSIS functions
// NVIC_SetPriority (IRQn, priority);
NVIC_SetPriority(7, 6); // IP=96
```


한동대학교 HANDONG GLOBAL UNIVERSITY

NVIC Registers

- **Software Trigger Interrupt Register (NVIC_STIR)**
  - Can trigger an interrupt with software

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved							INTID[8:0]								
							w	w	w	w	w	w	w	w	w

Bits 31:9 Reserved, must be kept cleared.

Bits 8:0 **INTID** Software generated interrupt ID

Write to the STIR to generate a Software Generated Interrupt (SGI). The value to be written is the Interrupt ID of the required SGI, in the range 0-239. For example, a value of 0x03 specifies interrupt IRQ3.

한동대학교 HANDONG GLOBAL UNIVERSITY